

F. Greetings

time limit per test: 5 seconds
memory limit per test: 256 megabytes

There are n people on the number line; the i -th person is at point a_i and wants to go to point b_i . For each person, $a_i < b_i$, and the starting and ending points of all people are distinct. (That is, all of the $2n$ numbers $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n$ are distinct.)

All the people will start moving simultaneously at a speed of 1 unit per second until they reach their final point b_i . When two people meet at the same point, they will greet each other once. How many greetings will there be?

Note that a person can still greet other people even if they have reached their final point.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of test cases follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of people.

Then n lines follow, the i -th of which contains two integers a_i and b_i ($-10^9 \leq a_i < b_i \leq 10^9$) — the starting and ending positions of each person.

For each test case, all of the $2n$ numbers $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n$ are distinct.

The sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer denoting the number of greetings that will happen.

Example

input	output
5	1
2	9
2 3	6
1 4	4
6	0
2 6	
3 9	
4 5	
1 8	
7 10	
-2 100	
4	
-10 10	
-5 5	
-12 12	
-13 13	
5	
-4 9	
-2 5	
3 4	
6 7	
8 10	
4	
1 2	
3 4	
5 6	
7 8	

Note

In the first test case, the two people will meet at point 3 and greet each other.

C. Smilo and Monsters

time limit per test: 1 second
memory limit per test: 256 megabytes

A boy called Smilo is playing a new game! In the game, there are n hordes of monsters, and the i -th horde contains a_i monsters. The goal of the game is to destroy all the monsters. To do this, you have two types of attacks and a combo counter x , initially set to 0:

- The first type: you choose a number i from 1 to n , such that there is at least one monster left in the horde with the number i . Then, you kill one monster from horde number i , and the combo counter x increases by 1.
- The second type: you choose a number i from 1 to n , such that there are at least x monsters left in the horde with number i . Then, you use an ultimate attack and kill x monsters from the horde with number i . After that, x is reset to zero.

Your task is to destroy all of the monsters, meaning that there should be no monsters left in any of the hordes. Smilo wants to win as quickly as possible, so he wants to the minimum number of attacks required to win the game.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The descriptions of the test cases follow.

The first line of each input data set contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of hordes of monsters.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the number of monsters in each horde.

It is guaranteed that the sum of n across all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, out put the minimum number of attacks required to kill all monsters.

Example

input	output
4	4
4	4
1 3 1 1	11
4	5
1 2 1 1	
6	
3 2 1 5 2 4	
2	
1 6	

Note

In the first test case, we can use an attack of the first type on the 1-st, 3-rd and 4-th hordes, and then use an attack of the second type to finish off the 2-nd horde. We need 4 attacks in total.

In the second test case, we can use an attack of the first type on the 1-st and 3-rd hordes, then use an attack of the second type to finish off the 2-nd horde, and then use an attack of the first type on the 4-th

horde. We need 4 attacks in total.

In the fourth test case, we can use an attack of the first type once on the 1-st horde, twice on the 2-nd horde, and then use an attack of the second type on the 2-nd horde, and finally use an attack of the first type to finish off the 2-nd horde. We need 5 attacks in total.

E. Block Sequence

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Given a sequence of integers a of length n .

A sequence is called *beautiful* if it has the form of a series of blocks, each starting with its length, i.e., first comes the length of the block, and then its elements. For example, the sequences [3, 3, 4, 5, 2, 6, 1] and [1, 8, 4, 5, 2, 6, 1] are *beautiful* (different blocks are colored differently), while [1], [1, 4, 3], [3, 2, 1] are not.

In one operation, you can remove any element from the sequence. What is the minimum number of operations required to make the given sequence *beautiful*?

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Then follows the descriptions of the test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the sequence a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$) — the elements of the sequence a .

It is guaranteed that the sum of the values of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single number — the minimum number of deletions required to make the sequence a *beautiful*.

Example

input	output
7	0
7	4
3 3 4 5 2 6 1	1
4	1
5 6 3 2	2
6	1
3 4 1 6 7 7	0
3	
1 4 3	
5	
1 2 3 4 5	
5	
1 2 3 1 2	
5	
4 5 5 1 5	

Note

In the first test case of the example, the given sequence is already *beautiful*, as shown in the statement.

In the second test case of the example, the sequence can only be made beautiful by removing all elements from it.

In the fifth test case of the example, the sequence can be made beautiful by removing the first and last elements. Then the sequence will become [2, 3, 4].

E. Data Structures Fan

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given an array of integers $a_1, a_2, \dots, a_n$, as well as a binary string[†] s consisting of n characters.

Augustin is a big fan of data structures. Therefore, he asked you to implement a data structure that can answer q queries. There are two types of queries:

- "1 l r " ($1 \leq l \leq r \leq n$) — replace each character s_i for $l \leq i \leq r$ with its opposite. That is, replace all 0 with 1 and all 1 with 0.
- "2 g " ($g \in \{0, 1\}$) — calculate the value of the [bitwise XOR](#) of the numbers a_i for all indices i such that $s_i = g$. Note that the XOR of an empty set of numbers is considered to be equal to 0.

Please help Augustin to answer all the queries!

For example, if $n = 4$, $a = [1, 2, 3, 6]$, $s = 1001$, consider the following series of queries:

1. "2 0" — we are interested in the indices i for which $s_i = 0$, since $s = 1001$, these are the indices 2 and 3, so the answer to the query will be $a_2 \oplus a_3 = 2 \oplus 3 = 1$.
2. "1 1 3" — we need to replace the characters s_1, s_2, s_3 with their opposites, so before the query $s = 1001$, and after the query: $s = 0111$.
3. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 0111$, these are the indices 2, 3, and 4, so the answer to the query will be $a_2 \oplus a_3 \oplus a_4 = 2 \oplus 3 \oplus 6 = 7$.
4. "1 2 4" — $s = 0111 \rightarrow s = 0000$.
5. "2 1" — $s = 0000$, there are no indices with $s_i = 1$, so since the XOR of an empty set of numbers is considered to be equal to 0, the answer to this query is 0.

[†] A binary string is a string containing only characters 0 or 1.

Input

The first line of the input contains one integer t ($1 \leq t \leq 10^4$) — the number of test cases in the test.

The descriptions of the test cases follow.

The first line of each test case description contains an integer n ($1 \leq n \leq 10^5$) — the length of the array.

The second line of the test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

The third line of the test case contains the binary string s of length n .

The fourth line of the test case contains one integer q ($1 \leq q \leq 10^5$) — the number of queries.

The subsequent q lines of the test case describe the queries. The first number of each query, $tp \in \{1, 2\}$, characterizes the type of the query: if $tp = 1$, then 2 integers $1 \leq l \leq r \leq n$ follow, meaning that the operation of type 1 should be performed with parameters l, r , and if $tp = 2$, then one integer $g \in \{0, 1\}$ follows, meaning that the operation of type 2 should be performed with parameter g .

It is guaranteed that the sum of n over all test cases does not exceed 10^5 , and also that the sum of q over all test cases does not exceed 10^5 .

Output

For each test case, and for each query of type 2 in it, output the answer to the corresponding query.

Example

input	output
5	3 2 6 7 7
5	11 7
1 2 3 4 5	0 0
01000	16430827
7	47
2 0	
2 1	
1 2 4	
2 0	
2 1	
1 1 3	
2 1	
6	
12 12 14 14 5 5	
001001	
3	
2 1	
1 2 4	
2 1	
4	
7 7 7 777	
1111	
3	
2 0	
1 2 3	
2 0	
2	
1000000000 996179179	
11	
1	
2 1	
5	
1 42 20 47 7	
00011	
5	
1 3 4	
1 1 1	
1 3 4	
1 2 4	
2 0	

Note

Let's analyze the first test case:

- "2 0" — we are interested in the indices i for which $s_i = 0$, since $s = 01000$, these are the indices 1, 3, 4, and 5, so the answer to the query will be $a_1 \oplus a_3 \oplus a_4 \oplus a_5 = 1 \oplus 3 \oplus 4 \oplus 5 = 3$.
- "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 01000$, the only suitable index is 2, so the answer to the query will be $a_2 = 2$.
- "1 2 4" — we need to replace the characters s_2, s_3, s_4 with their opposites, so before the query $s = 01000$, and after the query: $s = 00110$.

4. "2 0" — we are interested in the indices i for which $s_i = 0$, since $s = 00110$, these are the indices 1, 2, and 5, so the answer to the query will be $a_1 \oplus a_2 \oplus a_5 = 1 \oplus 2 \oplus 5 = 6$.
5. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 00110$, these are the indices 3 and 4, so the answer to the query will be $a_3 \oplus a_4 = 3 \oplus 4 = 7$.
6. "1 1 3" — $s = 00110 \rightarrow s = 11010$.
7. "2 1" — we are interested in the indices i for which $s_i = 1$, since $s = 11010$, these are the indices 1, 2, and 4, so the answer to the query will be $a_1 \oplus a_2 \oplus a_4 = 1 \oplus 2 \oplus 4 = 7$.

C. Tea Tasting

time limit per test: 2 seconds
memory limit per test: 256 megabytes

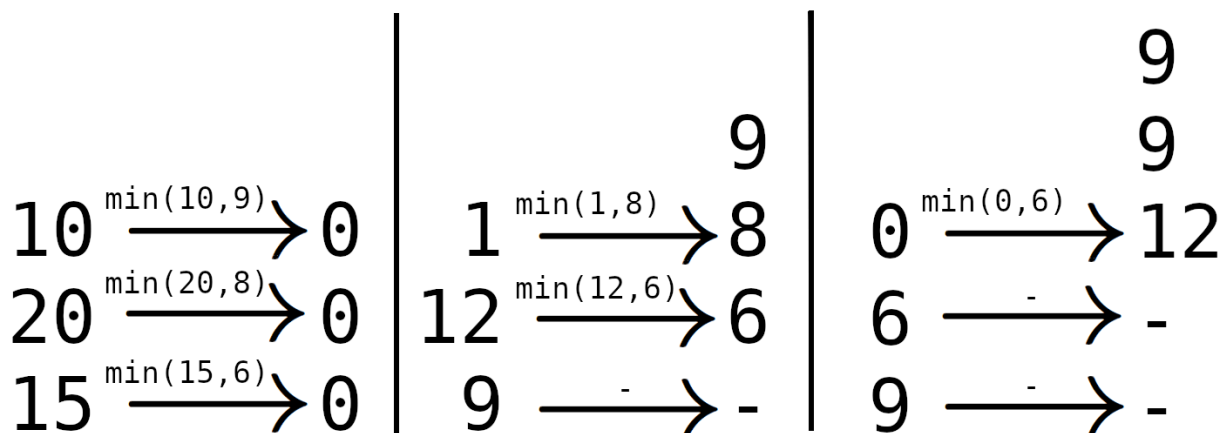
A tea manufacturer decided to conduct a massive tea tasting. n sorts of tea will be tasted by n tasters. Both the sorts of tea and the tasters are numbered from 1 to n . The manufacturer prepared a_i milliliters of the i -th sort of tea. The j -th taster can drink b_j milliliters of tea at once.

The tasting will be conducted in steps. During the first step, the i -th taster tastes the i -th sort of tea. The i -th taster drinks $\min(a_i, b_i)$ tea (how much is available of the i -th sort and how much the i -th taster can drink). a_i also decreases by this amount.

Then all tasters move to the previous sort of tea. Thus, during the second step, the i -th taster tastes the $(i - 1)$ -st sort of tea. The i -th taster drinks $\min(a_{i-1}, b_i)$ tea. The 1-st person ends the tasting.

During the third step, the i -th taster tastes the $(i - 2)$ -nd sort of tea. The 2-nd taster ends the tasting. This goes on until everyone ends the tasting.

Take a look at the tasting process for $n = 3$, $a = [10, 20, 15]$, $b = [9, 8, 6]$. In the left row, there are the current amounts of each sort of tea. In the right column, there are current amounts of tea each taster has drunk in total. The arrow tells which taster each tea goes to on the current step. The number on the arrow is the amount — minimum of how much is available of the sort of tea and how much the taster can drink.



For each taster, print how many milliliters of tea he/she will drink in total.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of testcases.

The first line of each testcase contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of sorts of tea and the number of tasters.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the amount of each sort of tea.

The third line contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the amount of tea each taster can drink at once.

The sum of n over all testcases doesn't exceed $2 \cdot 10^5$.

Output

For each testcase, print n integers — the i -th value should be equal to the total amount of tea the i -th taster will drink.

Example

input	output
4	9 9 12
3	5
10 20 15	3 8 6 4
9 8 6	1 2 2999999997
1	
5	
7	
4	
13 8 5 4	
3 4 2 1	
3	
10000000000 10000000000 10000000000	
1 1 10000000000	

Note

The first testcase is described in the statement. Here are the remaining amounts of each sort of tea *after* each step and the total amount of tea each taster has drunk:

- $a = [1, 12, 9]$, $ans = [9, 8, 6]$
- $a = [0, 6, 9]$, $ans = [9, 9, 12]$
- $a = [0, 6, 9]$, $ans = [9, 9, 12]$

In the second testcase, the only taster drinks $\min(5, 7)$ milliliters of tea of the only sort.

Here are the remaining amounts of each sort of tea *after* each step and the total amount of tea each taster has drunk for the third testcase:

- $a = [10, 4, 3, 3]$, $ans = [3, 4, 2, 1]$;
- $a = [6, 2, 2, 3]$, $ans = [3, 8, 4, 2]$;
- $a = [4, 1, 2, 3]$, $ans = [3, 8, 6, 3]$;
- $a = [3, 1, 2, 3]$, $ans = [3, 8, 6, 4]$.

Here are the remaining amounts of each sort of tea *after* each step and the total amount of tea each taster has drunk for the fourth testcase:

- $a = [999999999, 999999999, 0]$, $ans = [1, 1, 10000000000]$;
- $a = [999999998, 0, 0]$, $ans = [1, 2, 1999999999]$;
- $a = [0, 0, 0]$, $ans = [1, 2, 2999999997]$.

L. Controllers

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are at your grandparents' house and you are playing an old video game on a strange console. Your controller has only two buttons and each button has a number written on it.

Initially, your score is 0. The game is composed of n rounds. For each $1 \leq i \leq n$, the i -th round works as follows.

On the screen, a symbol s_i appears, which is either $+$ (*plus*) or $-$ (*minus*). Then you must press one of the two buttons on the controller **once**. Suppose you press a button with the number x written on it: your score will increase by x if the symbol was $+$ and will decrease by x if the symbol was $-$. After you press the button, the round ends.

After you have played all n rounds, you win if your score is 0.

Over the years, your grandparents bought many different controllers, so you have q of them. The two buttons on the j -th controller have the numbers a_j and b_j written on them. For each controller, you must compute whether you can win the game playing with that controller.

Input

The first line contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of rounds.

The second line contains a string s of length n — where s_i is the symbol that will appear on the screen in the i -th round. It is guaranteed that s contains only the characters $+$ and $-$.

The third line contains an integer q ($1 \leq q \leq 10^5$) — the number of controllers.

The following q lines contain two integers a_j and b_j each ($1 \leq a_j, b_j \leq 10^9$) — the numbers on the buttons of controller j .

Output

Output q lines. On line j print **YES** if the game is winnable using controller j , otherwise print **NO**.

Examples

input	output
8 +-+-----+ 5 2 1 10 3 7 9 10 10 5 3	YES NO NO NO YES
input	output
6 +-++-- 2 9 7 1 1	YES YES

input	output
20 +-----+--+-----+- 2 1000000000 99999997 250000000 1000000000	NO YES

Note

In the **first sample**, one possible way to get score 0 using the first controller is by pressing the button with number 1 in rounds 1, 2, 4, 5, 6 and 8, and pressing the button with number 2 in rounds 3 and 7. It is possible to show that there is no way to get a score of 0 using the second controller.

C. Palindrome Basis

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given a positive integer n . Let's call some positive integer a without leading zeroes palindromic if it remains the same after reversing the order of its digits. Find the number of distinct ways to express n as a sum of positive palindromic integers. Two ways are considered different if the frequency of at least one palindromic integer is different in them. For example, $5 = 4 + 1$ and $5 = 3 + 1 + 1$ are considered different but $5 = 3 + 1 + 1$ and $5 = 1 + 3 + 1$ are considered the same.

Formally, you need to find the number of distinct multisets of positive palindromic integers the sum of which is equal to n .

Since the answer can be quite large, print it modulo $10^9 + 7$.

Input

The first line of input contains a single integer t ($1 \leq t \leq 10^4$) denoting the number of testcases.

Each testcase contains a single line of input containing a single integer n ($1 \leq n \leq 4 \cdot 10^4$) — the required sum of palindromic integers.

Output

For each testcase, print a single integer denoting the required answer modulo $10^9 + 7$.

Example

input	output
2	7
5	74
12	

Note

For the first testcase, there are 7 ways to partition 5 as a sum of positive palindromic integers:

- $5 = 1 + 1 + 1 + 1 + 1$
- $5 = 1 + 1 + 1 + 2$
- $5 = 1 + 2 + 2$
- $5 = 1 + 1 + 3$
- $5 = 2 + 3$
- $5 = 1 + 4$
- $5 = 5$

For the second testcase, there are total 77 ways to partition 12 as a sum of positive integers but among them, the partitions $12 = 2 + 10$, $12 = 1 + 1 + 10$ and $12 = 12$ are not valid partitions of 12 as a sum of positive palindromic integers because 10 and 12 are not palindromic. So, there are 74 ways to partition 12 as a sum of positive palindromic integers.

C. Line Empire

time limit per test: 1 second
memory limit per test: 256 megabytes

You are an ambitious king who wants to be the Emperor of The Reals. But to do that, you must first become Emperor of The Integers.

Consider a number axis. The capital of your empire is initially at 0. There are n unconquered kingdoms at positions $0 < x_1 < x_2 < \dots < x_n$. You want to conquer all other kingdoms.

There are two actions available to you:

- You can change the location of your capital (let its current position be c_1) to any other **conquered** kingdom (let its position be c_2) at a cost of $a \cdot |c_1 - c_2|$.
- From the current capital (let its current position be c_1) you can conquer an unconquered kingdom (let its position be c_2) at a cost of $b \cdot |c_1 - c_2|$. You **cannot** conquer a kingdom if there is an unconquered kingdom between the target and your capital.

Note that you **cannot** place the capital at a point without a kingdom. In other words, at any point, your capital can only be at 0 or one of $x_1, x_2, \dots, x_n$. Also note that conquering a kingdom does not change the position of your capital.

Find the minimum total cost to conquer all kingdoms. Your capital can be anywhere at the end.

Input

The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases. The description of each test case follows.

The first line of each test case contains 3 integers n , a , and b ($1 \leq n \leq 2 \cdot 10^5$; $1 \leq a, b \leq 10^5$).

The second line of each test case contains n integers $x_1, x_2, \dots, x_n$ ($1 \leq x_1 < x_2 < \dots < x_n \leq 10^8$).

The sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum cost to conquer all kingdoms.

Example

input	output
4	173
5 2 7	171
3 5 12 13 21	75
5 6 3	3298918744
1 5 6 21 30	
2 9 3	
10 15	
11 27182 31415	
16 18 33 98 874 989 4848 20458 34365	
38117 72030	

Note

Here is an optimal sequence of moves for the second test case:

1. Conquer the kingdom at position 1 with cost $3 \cdot (1 - 0) = 3$.
2. Move the capital to the kingdom at position 1 with cost $6 \cdot (1 - 0) = 6$.
3. Conquer the kingdom at position 5 with cost $3 \cdot (5 - 1) = 12$.
4. Move the capital to the kingdom at position 5 with cost $6 \cdot (5 - 1) = 24$.
5. Conquer the kingdom at position 6 with cost $3 \cdot (6 - 5) = 3$.
6. Conquer the kingdom at position 21 with cost $3 \cdot (21 - 5) = 48$.
7. Conquer the kingdom at position 30 with cost $3 \cdot (30 - 5) = 75$.

The total cost is $3 + 6 + 12 + 24 + 3 + 48 + 75 = 171$. You cannot get a lower cost than this.

C. Factorials and Powers of Two

time limit per test: 3 seconds
memory limit per test: 256 megabytes

A number is called *powerful* if it is a power of two or a factorial. In other words, the number m is powerful if there exists a non-negative integer d such that $m = 2^d$ or $m = d!$, where $d! = 1 \cdot 2 \cdot \dots \cdot d$ (in particular, $0! = 1$). For example 1, 4, and 6 are powerful numbers, because $1 = 1!$, $4 = 2^2$, and $6 = 3!$ but 7, 10, or 18 are not.

You are given a positive integer n . Find the minimum number k such that n can be represented as the sum of k **distinct** powerful numbers, or say that there is no such k .

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). Description of the test cases follows.

A test case consists of only one line, containing one integer n ($1 \leq n \leq 10^{12}$).

Output

For each test case print the answer on a separate line.

If n can not be represented as the sum of **distinct** powerful numbers, print -1 .

Otherwise, print a single positive integer — the minimum possible value of k .

Example

input	output
4	2
7	3
11	4
240	1
17179869184	

Note

In the first test case, 7 can be represented as $7 = 1 + 6$, where 1 and 6 are powerful numbers. Because 7 is not a powerful number, we know that the minimum possible value of k in this case is $k = 2$.

In the second test case, a possible way to represent 11 as the sum of three powerful numbers is $11 = 1 + 4 + 6$. We can show that there is no way to represent 11 as the sum of two or less powerful numbers.

In the third test case, 240 can be represented as $240 = 24 + 32 + 64 + 120$. Observe that $240 = 120 + 120$ is not a valid representation, because the powerful numbers have to be **distinct**.

In the fourth test case, $17179869184 = 2^{34}$, so 17179869184 is a powerful number and the minimum k in this case is $k = 1$.

B. AGAGA XOOORRR

time limit per test: 1 second
memory limit per test: 256 megabytes

Baby Ehab is known for his love for a certain operation. He has an array a of length n , and he decided to keep doing the following operation on it:

- he picks 2 adjacent elements; he then removes them and places a single integer in their place: their [bitwise XOR](#). Note that the length of the array decreases by one.

Now he asks you if he can make all elements of the array equal. Since babies like to make your life harder, he requires that you leave at least 2 elements remaining.

Input

The first line contains an integer t ($1 \leq t \leq 15$) — the number of test cases you need to solve.

The first line of each test case contains an integers n ($2 \leq n \leq 2000$) — the number of elements in the array a .

The second line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$) — the elements of the array a .

Output

If Baby Ehab can make all elements equal while leaving at least 2 elements standing, print "YES". Otherwise, print "NO".

Example

input	output
2	YES
3	NO
0 2 2	
4	
2 3 1 10	

Note

In the first sample, he can remove the first 2 elements, 0 and 2, and replace them by $0 \oplus 2 = 2$. The array will be $[2, 2]$, so all the elements are equal.

In the second sample, there's no way to make all the elements equal.

B. Eastern Exhibition

time limit per test: 1 second
memory limit per test: 256 megabytes

You and your friends live in n houses. Each house is located on a 2D plane, in a point with integer coordinates. There might be different houses located in the same point. The mayor of the city is asking you for places for the building of the Eastern exhibition. You have to find the number of places (points with integer coordinates), so that the summary distance from all the houses to the exhibition is minimal. The exhibition can be built in the same point as some house. The distance between two points (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$, where $|x|$ is the absolute value of x .

Input

First line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 1000$). Next n lines describe the positions of the houses (x_i, y_i) ($0 \leq x_i, y_i \leq 10^9$).

It's guaranteed that the sum of all n does not exceed 1000.

Output

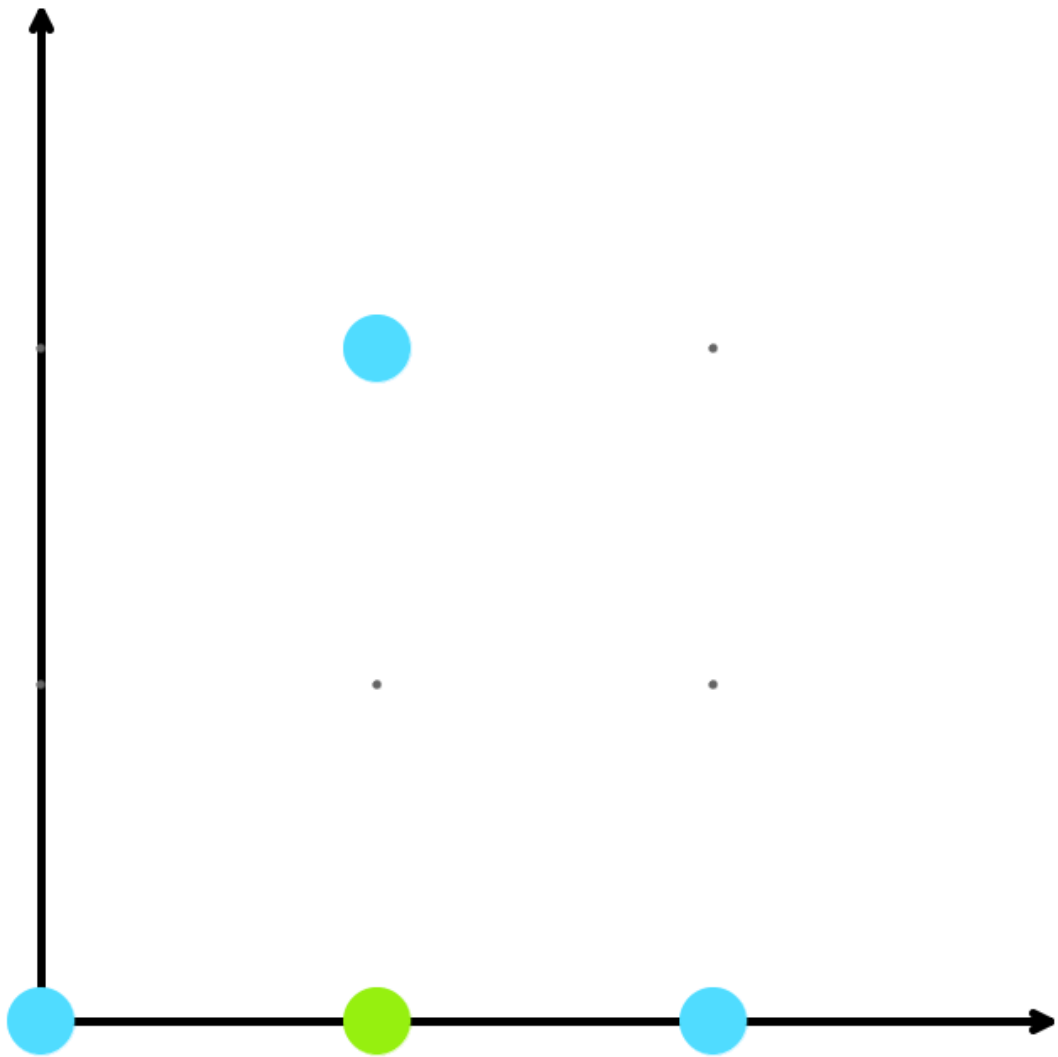
For each test case output a single integer - the number of different positions for the exhibition. The exhibition can be built in the same point as some house.

Example

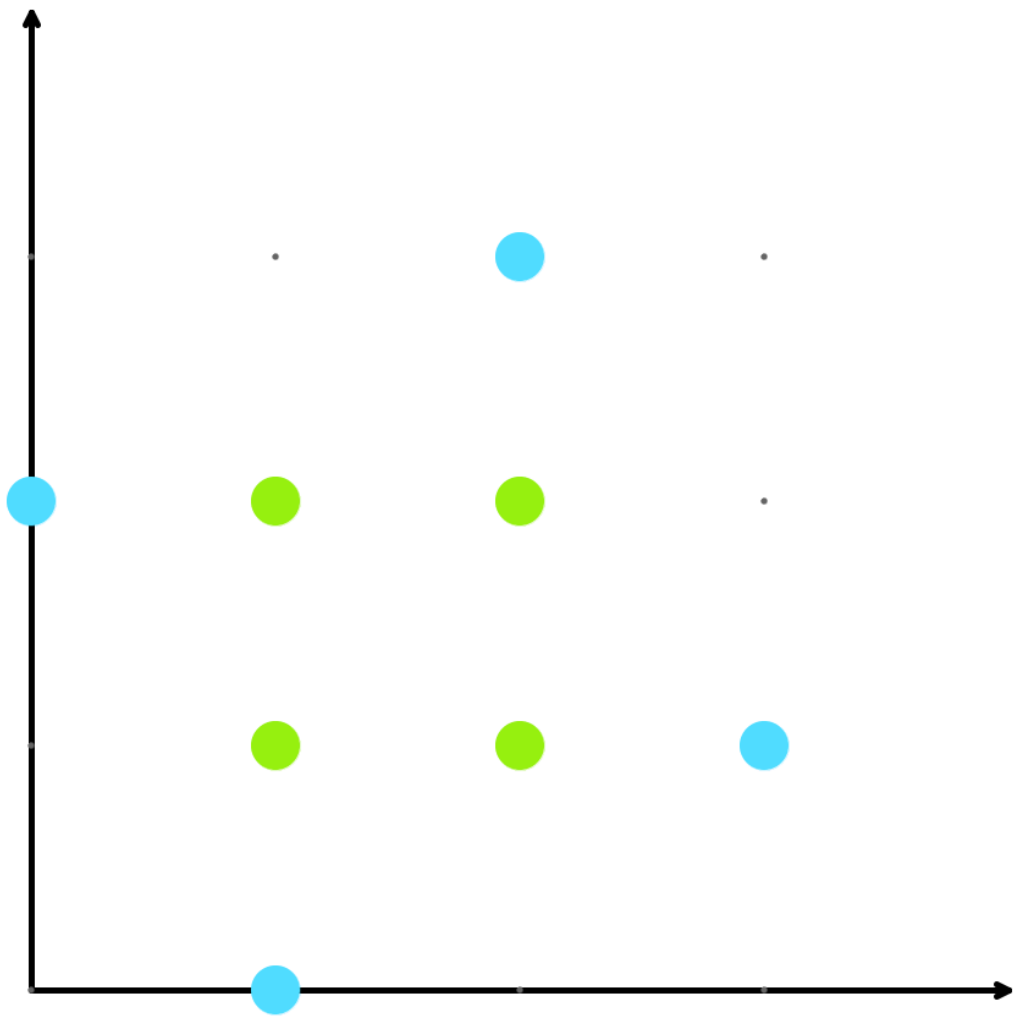
input	output
6	1
3	4
0 0	4
2 0	4
1 2	3
4	1
1 0	
0 2	
2 3	
3 1	
4	
0 0	
0 1	
1 0	
1 1	
2	
0 0	
1 1	
2	
0 0	
2 0	
2	
0 0	
0 0	

Note

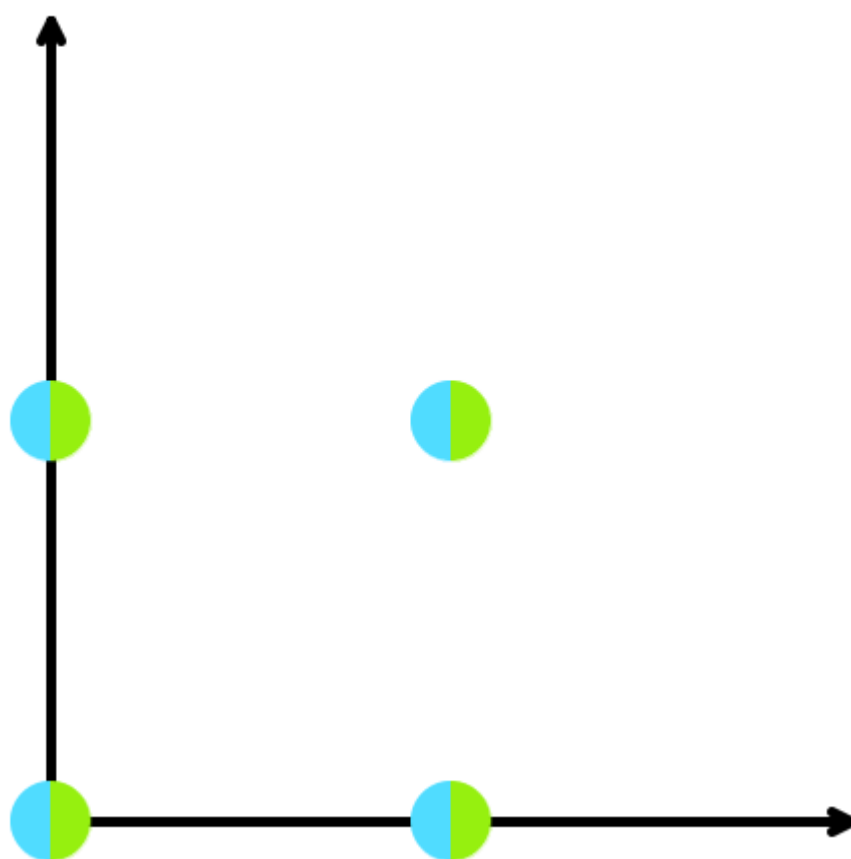
Here are the images for the example test cases. Blue dots stand for the houses, green — possible positions for the exhibition.



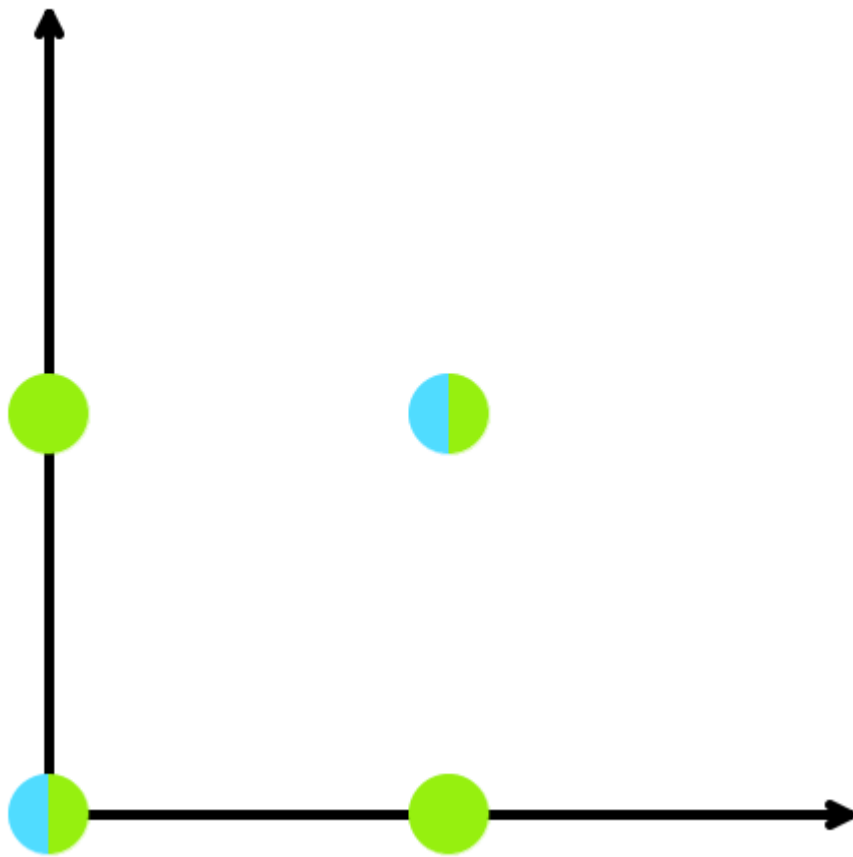
First test case.



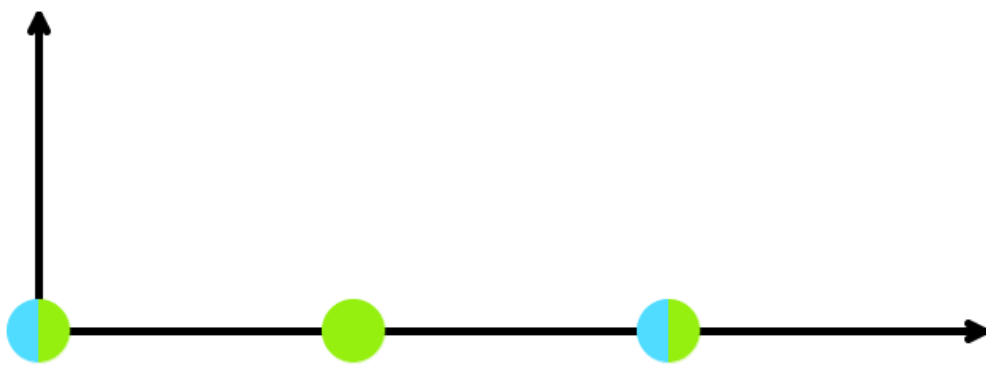
Second test case.



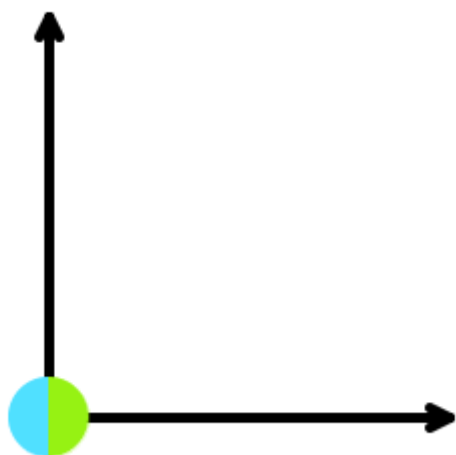
Third test case.



Fourth test case.



Fifth test case.



Sixth test case. Here both houses are located at $(0, 0)$.

D. 13th Labour of Heracles

time limit per test: 2.5 seconds

memory limit per test: 256 megabytes

You've probably heard about the twelve labors of Heracles, but do you have any idea about the thirteenth? It is commonly assumed it took him a dozen years to complete the twelve feats, so on average, a year to accomplish every one of them. As time flows faster these days, you have minutes rather than months to solve this task. But will you manage?

In this problem, you are given a tree with n weighted vertices. A tree is a connected graph with $n - 1$ edges.

Let us define its k -coloring as an assignment of k colors to the edges so that each edge has exactly one color assigned to it. Note that you don't have to use all k colors.

A subgraph of color x consists of these edges from the original tree, which are assigned color x , and only those vertices that are adjacent to at least one such edge. So there are no vertices of degree 0 in such a subgraph.

The value of a connected component is the **sum** of weights of its vertices. Let us define the value of a subgraph as a **maximum** of values of its connected components. We will assume that the value of an empty subgraph equals 0.

There is also a value of a k -coloring, which equals the **sum** of values of subgraphs of all k colors. Given a tree, for each k from 1 to $n - 1$ calculate the maximal value of a k -coloring.

Input

In the first line of input, there is a single integer t ($1 \leq t \leq 10^5$) denoting the number of test cases. Then t test cases follow.

First line of each test case contains a single integer n ($2 \leq n \leq 10^5$). The second line consists of n integers $w_1, w_2, \dots, w_n$ ($0 \leq w_i \leq 10^9$), w_i equals the weight of i -th vertex. In each of the following $n - 1$ lines, there are two integers u, v ($1 \leq u, v \leq n$) describing an edge between vertices u and v . It is guaranteed that these edges form a tree.

The sum of n in all test cases will not exceed $2 \cdot 10^5$.

Output

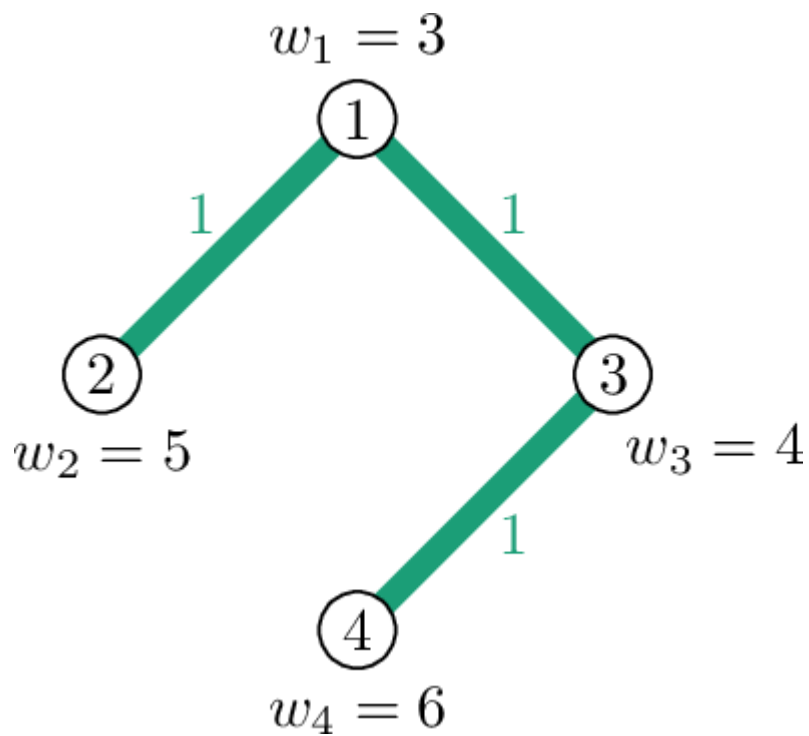
For every test case, your program should print one line containing $n - 1$ integers separated with a single space. The i -th number in a line should be the maximal value of a i -coloring of the tree.

Example

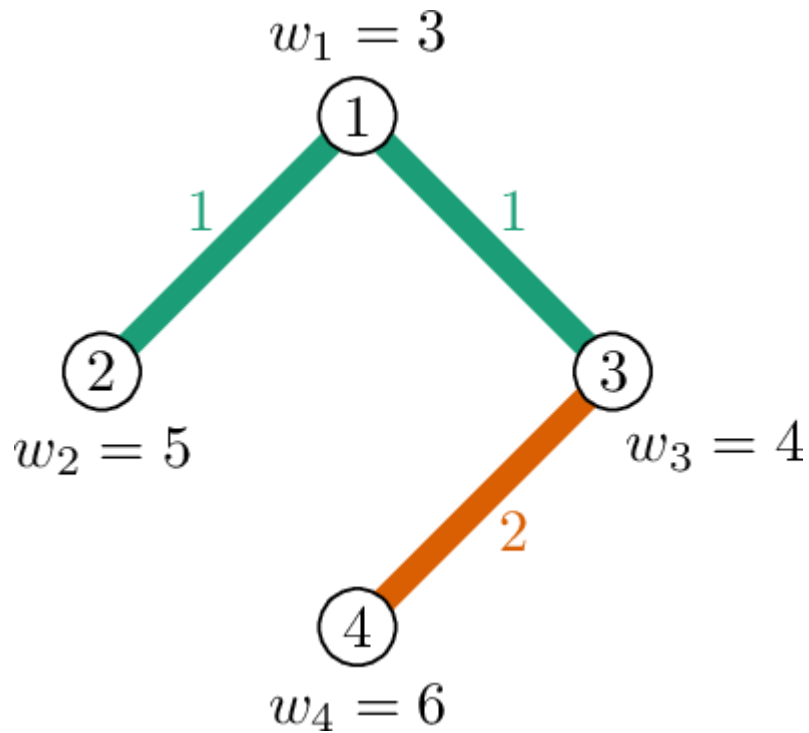
input	output
4	18 22 25
4	53
3 5 4 6	87 107 127 147 167
2 1	28 38 44
3 1	
4 3	
2	
21 32	
2 1	
6	
20 13 17 13 13 11	
2 1	
3 1	
4 1	
5 1	
6 1	
4	
10 6 6 6	
1 2	
2 3	
4 1	

Note

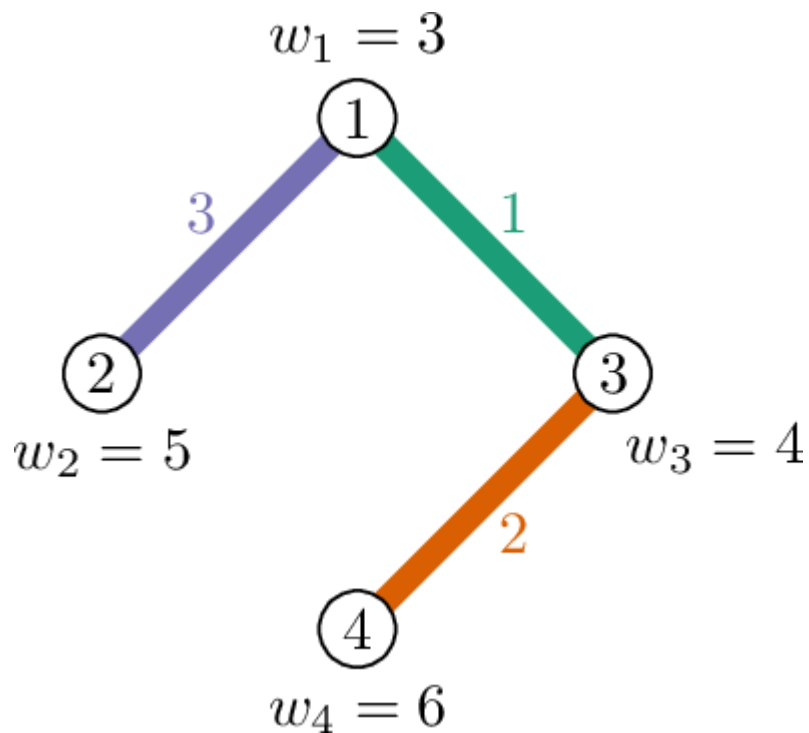
The optimal k -colorings from the first test case are the following:



In the 1-coloring all edges are given the same color. The subgraph of color 1 contains all the edges and vertices from the original graph. Hence, its value equals $3 + 5 + 4 + 6 = 18$.



In an optimal 2-coloring edges $(2, 1)$ and $(3, 1)$ are assigned color 1. Edge $(4, 3)$ is of color 2. Hence the subgraph of color 1 consists of a single connected component (vertices 1, 2, 3) and its value equals $3 + 5 + 4 = 12$. The subgraph of color 2 contains two vertices and one edge. Its value equals $4 + 6 = 10$.



In an optimal 3-coloring all edges are assigned distinct colors. Hence subgraphs of each color consist of a single edge. Their values are as follows: $3 + 4 = 7$, $4 + 6 = 10$, $3 + 5 = 8$.

C. Mortal Kombat Tower

time limit per test: 1 second

memory limit per test: 256 megabytes

You and your friend are playing the game Mortal Kombat XI. You are trying to pass a challenge tower. There are n bosses in this tower, numbered from 1 to n . The type of the i -th boss is a_i . If the i -th boss is easy then its type is $a_i = 0$, otherwise this boss is hard and its type is $a_i = 1$.

During one session, either you or your friend can kill **one or two** bosses (neither you nor your friend can skip the session, so the minimum number of bosses killed during one session is at least one). After your friend session, your session begins, then again your friend session begins, your session begins, and so on. **The first session is your friend's session.**

Your friend needs to get good because he can't actually kill hard bosses. To kill them, he uses skip points. One skip point can be used to kill one hard boss.

Your task is to find the **minimum** number of skip points your friend needs to use so you and your friend kill all n bosses in the given order.

For example: suppose $n = 8$, $a = [1, 0, 1, 1, 0, 1, 1, 1]$. Then the best course of action is the following:

- your friend kills two first bosses, using one skip point for the first boss;
- you kill the third and the fourth bosses;
- your friend kills the fifth boss;
- you kill the sixth and the seventh bosses;
- your friend kills the last boss, using one skip point, so the tower is completed using two skip points.

You have to answer t independent test cases.

Input

The first line of the input contains one integer t ($1 \leq t \leq 2 \cdot 10^4$) — the number of test cases. Then t test cases follow.

The first line of the test case contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of bosses. The second line of the test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 1$), where a_i is the type of the i -th boss.

It is guaranteed that the sum of n does not exceed $2 \cdot 10^5$ ($\sum n \leq 2 \cdot 10^5$).

Output

For each test case, print the answer: the **minimum** number of skip points your friend needs to use so you and your friend kill all n bosses in the given order.

Example

input	output
6	2
8	2
1 0 1 1 0 1 1 1	2
5	2
1 1 1 1 0	1
7	0
1 1 1 1 0 0 1	
6	
1 1 1 1 1 1	
1	
1	
1	
0	

C. k-Amazing Numbers

time limit per test: 1 second
memory limit per test: 256 megabytes

You are given an array a consisting of n integers numbered from 1 to n .

Let's define the k -amazing number of the array as the minimum number that occurs in all of the subsegments of the array having length k (recall that a subsegment of a of length k is a contiguous part of a containing exactly k elements). If there is no integer occurring in all subsegments of length k for some value of k , then the k -amazing number is -1 .

For each k from 1 to n calculate the k -amazing number of the array a .

Input

The first line contains one integer t ($1 \leq t \leq 1000$) — the number of test cases. Then t test cases follow.

The first line of each test case contains one integer n ($1 \leq n \leq 3 \cdot 10^5$) — the number of elements in the array. The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case print n integers, where the i -th integer is equal to the i -amazing number of the array.

Example

input	output
3	-1 -1 3 2 1
5	-1 4 4 4 2
1 2 3 4 5	-1 -1 1 1 1 1
5	
4 4 4 4 2	
6	
1 3 1 5 3 1	

C. Balanced Bitstring

time limit per test: 2 seconds
memory limit per test: 256 megabytes

A bitstring is a string consisting only of the characters 0 and 1. A bitstring is called k -**balanced** if every substring of size k of this bitstring has an equal amount of 0 and 1 characters ($\frac{k}{2}$ of each).

You are given an integer k and a string s which is composed only of characters 0, 1, and ?. You need to determine whether you can make a k -balanced bitstring by replacing every ? characters in s with either 0 or 1.

A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). Description of the test cases follows.

The first line of each test case contains two integers n and k ($2 \leq k \leq n \leq 3 \cdot 10^5$, k is even) — the length of the string and the parameter for a balanced bitstring.

The next line contains the string s ($|s| = n$). It is given that s consists of only 0, 1, and ?.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print YES if we can replace every ? in s with 0 or 1 such that the resulting bitstring is k -balanced, or NO if it is not possible.

Example

input	output
9	YES
6 4	YES
100110	NO
3 2	YES
1?1	YES
3 2	NO
1?0	NO
4 4	YES
????	NO
7 4	
1?0???1?	
10 10	
11???11???11	
4 2	
1???1	
4 4	
?0?0	
6 2	
????00	

Note

For the first test case, the string is already a 4-balanced bitstring.

For the second test case, the string can be transformed into 101.

For the fourth test case, the string can be transformed into 0110.

For the fifth test case, the string can be transformed into 1100110.

A. Powered Addition

time limit per test: 1 second
memory limit per test: 256 megabytes

You have an array a of length n . For every positive integer x you are going to perform the following operation during the x -th second:

- Select some distinct indices $i_1, i_2, \dots, i_k$ which are between 1 and n inclusive, and add 2^{x-1} to each corresponding position of a . Formally, $a_{i_j} := a_{i_j} + 2^{x-1}$ for $j = 1, 2, \dots, k$. **Note that you are allowed to not select any indices at all.**

You have to make a nondecreasing as fast as possible. Find the smallest number T such that you can make the array nondecreasing after at most T seconds.

Array a is nondecreasing if and only if $a_1 \leq a_2 \leq \dots \leq a_n$.

You have to answer t independent test cases.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains single integer n ($1 \leq n \leq 10^5$) — the length of array a . It is guaranteed that the sum of values of n over all test cases in the input does not exceed 10^5 .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$).

Output

For each test case, print the minimum number of seconds in which you can make a nondecreasing.

Example

input	output
3	2
4	0
1 7 6 5	3
5	
1 2 3 4 5	
2	
0 -4	

Note

In the first test case, if you select indices 3, 4 at the 1-st second and 4 at the 2-nd second, then a will become $[1, 7, 7, 8]$. There are some other possible ways to make a nondecreasing in 2 seconds, but you can't do it faster.

In the second test case, a is already nondecreasing, so answer is 0.

In the third test case, if you do nothing at first 2 seconds and select index 2 at the 3-rd second, a will become $[0, 0]$.

C. K-Complete Word

time limit per test: 2 seconds
memory limit per test: 512 megabytes

Word s of length n is called k -complete if

- s is a palindrome, i.e. $s_i = s_{n+1-i}$ for all $1 \leq i \leq n$;
- s has a period of k , i.e. $s_i = s_{k+i}$ for all $1 \leq i \leq n - k$.

For example, "abaaba" is a 3-complete word, while "abccba" is not.

Bob is given a word s of length n consisting of only lowercase Latin letters and an integer k , such that n is divisible by k . He wants to convert s to any k -complete word.

To do this Bob can choose some i ($1 \leq i \leq n$) and replace the letter at position i with some other lowercase Latin letter.

So now Bob wants to know the minimum number of letters he has to replace to convert s to any k -complete word.

Note that Bob can do zero changes if the word s is already k -complete.

You are required to answer t test cases **independently**.

Input

The first line contains a single integer t ($1 \leq t \leq 10^5$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq k < n \leq 2 \cdot 10^5$, n is divisible by k).

The second line of each test case contains a word s of length n .

It is guaranteed that word s only contains lowercase Latin letters. And it is guaranteed that the sum of n over all test cases will not exceed $2 \cdot 10^5$.

Output

For each test case, output one integer, representing the minimum number of characters he has to replace to convert s to any k -complete word.

Example

input	output
4	2
6 2	0
abaaba	23
6 3	16
abaaba	
36 9	
hippopotomonstrosesquippedaliophobia	
21 7	
wudixiaoxingxingheclp	

Note

In the first test case, one optimal solution is aaaaaa.

In the second test case, the given word itself is k -complete.

C. Ehab and Path-etic MEXs

time limit per test: 1 second

memory limit per test: 256 megabytes

You are given a tree consisting of n nodes. You want to write some labels on the tree's edges such that the following conditions hold:

- Every label is an integer between 0 and $n - 2$ inclusive.
- All the written labels are distinct.
- The largest value among $MEX(u, v)$ over all pairs of nodes (u, v) is as small as possible.

Here, $MEX(u, v)$ denotes the smallest non-negative integer that isn't written on any edge on the unique simple path from node u to node v .

Input

The first line contains the integer n ($2 \leq n \leq 10^5$) — the number of nodes in the tree.

Each of the next $n - 1$ lines contains two space-separated integers u and v ($1 \leq u, v \leq n$) that mean there's an edge between nodes u and v . It's guaranteed that the given graph is a tree.

Output

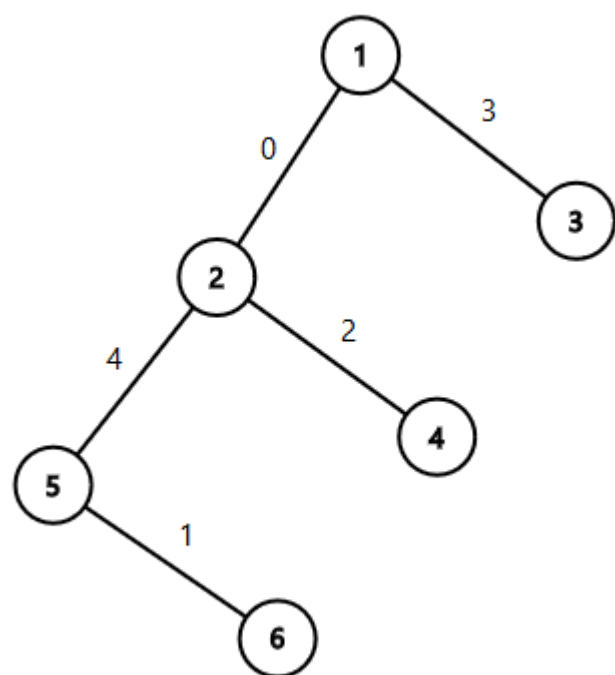
Output $n - 1$ integers. The i^{th} of them will be the number written on the i^{th} edge (in the input order).

Examples

input	output
3 1 2 1 3	0 1
input	output
6 1 2 1 3 2 4 2 5 5 6	0 3 2 4 1

Note

The tree from the second sample:



B. Count Subrectangles

time limit per test: 1 second
memory limit per test: 512 megabytes

You are given an array a of length n and array b of length m both consisting of only integers 0 and 1. Consider a matrix c of size $n \times m$ formed by following rule: $c_{i,j} = a_i \cdot b_j$ (i.e. a_i multiplied by b_j). It's easy to see that c consists of only zeroes and ones too.

How many *subrectangles* of size (area) k consisting only of ones are there in c ?

A *subrectangle* is an intersection of a consecutive (subsequent) segment of rows and a consecutive (subsequent) segment of columns. I.e. consider four integers x_1, x_2, y_1, y_2 ($1 \leq x_1 \leq x_2 \leq n$, $1 \leq y_1 \leq y_2 \leq m$) a subrectangle $c[x_1 \dots x_2][y_1 \dots y_2]$ is an intersection of the rows $x_1, x_1 + 1, x_1 + 2, \dots, x_2$ and the columns $y_1, y_1 + 1, y_1 + 2, \dots, y_2$.

The size (area) of a subrectangle is the total number of cells in it.

Input

The first line contains three integers n, m and k ($1 \leq n, m \leq 40\,000, 1 \leq k \leq n \cdot m$), length of array a , length of array b and required size of subrectangles.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 1$), elements of a .

The third line contains m integers $b_1, b_2, \dots, b_m$ ($0 \leq b_i \leq 1$), elements of b .

Output

Output single integer — the number of subrectangles of c with size (area) k consisting only of ones.

Examples

input	output
3 3 2 1 0 1 1 1 1	4
input	output
3 5 4 1 1 1 1 1 1 1 1	14

Note

In first example matrix c is:

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

There are 4 subrectangles of size 2 consisting of only ones in it:

$$\begin{pmatrix} \boxed{1} & \boxed{1} & 1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix} \quad
 \begin{pmatrix} 1 & \boxed{1} & \boxed{1} \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix} \quad
 \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ \boxed{1} & \boxed{1} & 1 \end{pmatrix} \quad
 \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & \boxed{1} & \boxed{1} \end{pmatrix}$$

In second example matrix c is:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

B. Zero Array

time limit per test: 1 second
memory limit per test: 256 megabytes

You are given an array $a_1, a_2, \dots, a_n$.

In one operation you can choose two elements a_i and a_j ($i \neq j$) and decrease each of them by one.

You need to check whether it is possible to make all the elements equal to zero or not.

Input

The first line contains a single integer n ($2 \leq n \leq 10^5$) — the size of the array.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array.

Output

Print "YES" if it is possible to make all elements zero, otherwise print "NO".

Examples

input	output
4 1 1 2 2	YES
input	output
6 1 2 3 4 5 6	NO

Note

In the first example, you can make all elements equal to zero in 3 operations:

- Decrease a_1 and a_2 ,
- Decrease a_3 and a_4 ,
- Decrease a_3 and a_4

In the second example, one can show that it is impossible to make all elements equal to zero.

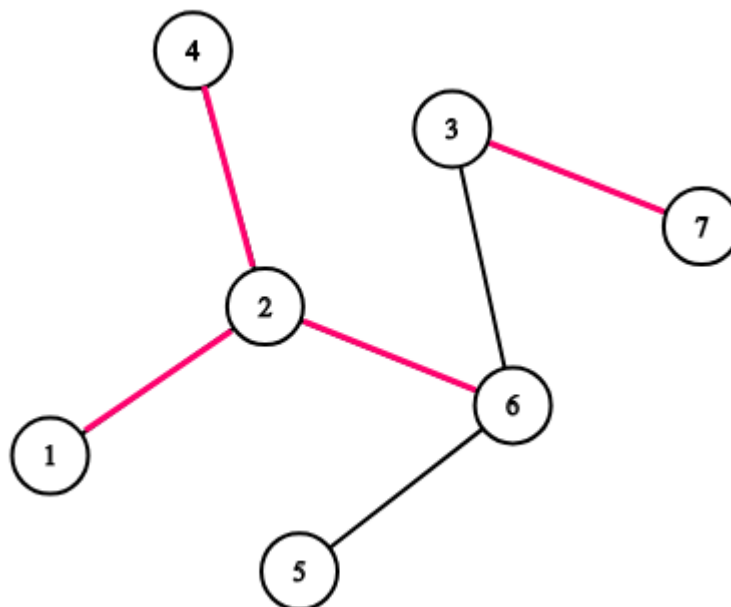
C. Edgy Trees

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given a tree (a connected undirected graph without cycles) of n vertices. Each of the $n - 1$ edges of the tree is colored in either black or red.

You are also given an integer k . Consider sequences of k vertices. Let's call a sequence $[a_1, a_2, \dots, a_k]$ *good* if it satisfies the following criterion:

- We will walk a path (possibly visiting same edge/vertex multiple times) on the tree, starting from a_1 and ending at a_k .
- Start at a_1 , then go to a_2 using the shortest path between a_1 and a_2 , then go to a_3 in a similar way, and so on, until you travel the shortest path between a_{k-1} and a_k .
- If you walked over at least one black edge during this process, then the sequence is good.



Consider the tree on the picture. If $k = 3$ then the following sequences are good: $[1, 4, 7]$, $[5, 5, 3]$ and $[2, 3, 7]$. The following sequences are not good: $[1, 4, 6]$, $[5, 5, 5]$, $[3, 7, 3]$.

There are n^k sequences of vertices, count how many of them are good. Since this number can be quite large, print it modulo $10^9 + 7$.

Input

The first line contains two integers n and k ($2 \leq n \leq 10^5$, $2 \leq k \leq 100$), the size of the tree and the length of the vertex sequence.

Each of the next $n - 1$ lines contains three integers u_i , v_i and x_i ($1 \leq u_i, v_i \leq n$, $x_i \in \{0, 1\}$), where u_i and v_i denote the endpoints of the corresponding edge and x_i is the color of this edge (0 denotes red edge and 1 denotes black edge).

Output

Print the number of good sequences modulo $10^9 + 7$.

Examples

input	output
4 4 1 2 1 2 3 1 3 4 1	252
input	output
4 6 1 2 0 1 3 0 1 4 0	0
input	output
3 5 1 2 1 2 3 0	210

Note

In the first example, all sequences (4^4) of length 4 **except** the following are good:

- [1, 1, 1, 1]
- [2, 2, 2, 2]
- [3, 3, 3, 3]
- [4, 4, 4, 4]

In the second example, all edges are red, hence there aren't any good sequences.

D. Zero Quantity Maximization

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given two arrays a and b , each contains n integers.

You want to create a new array c as follows: choose some real (i.e. not necessarily integer) number d , and then for every $i \in [1, n]$ let $c_i := d \cdot a_i + b_i$.

Your goal is to maximize the number of zeroes in array c . What is the largest possible answer, if you choose d optimally?

Input

The first line contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of elements in both arrays.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$).

The third line contains n integers $b_1, b_2, \dots, b_n$ ($-10^9 \leq b_i \leq 10^9$).

Output

Print one integer — the maximum number of zeroes in array c , if you choose d optimally.

Examples

input 5 1 2 3 4 5 2 4 7 11 3	output 2
input 3 13 37 39 1 2 3	output 2
input 4 0 0 0 0 1 2 3 4	output 0
input 3 1 2 -1 -6 -12 6	output 3

Note

In the first example, we may choose $d = -2$.

In the second example, we may choose $d = -\frac{1}{13}$.

In the third example, we cannot obtain any zero in array c , no matter which d we choose.

In the fourth example, we may choose $d = 6$.

D. Lunar New Year and a Wander

time limit per test: 3 seconds
memory limit per test: 256 megabytes

Lunar New Year is approaching, and Bob decides to take a wander in a nearby park.

The park can be represented as a connected graph with n nodes and m bidirectional edges. Initially Bob is at the node 1 and he records 1 on his notebook. He can wander from one node to another through those bidirectional edges. Whenever he visits a node not recorded on his notebook, he records it. After he visits all nodes at least once, he stops wandering, thus finally a permutation of nodes $a_1, a_2, \dots, a_n$ is recorded.

Wandering is a boring thing, but solving problems is fascinating. Bob wants to know the lexicographically smallest sequence of nodes he can record while wandering. Bob thinks this problem is trivial, and he wants you to solve it.

A sequence x is lexicographically smaller than a sequence y if and only if one of the following holds:

- x is a prefix of y , but $x \neq y$ (this is impossible in this problem as all considered sequences have the same length);
- in the first position where x and y differ, the sequence x has a smaller element than the corresponding element in y .

Input

The first line contains two positive integers n and m ($1 \leq n, m \leq 10^5$), denoting the number of nodes and edges, respectively.

The following m lines describe the bidirectional edges in the graph. The i -th of these lines contains two integers u_i and v_i ($1 \leq u_i, v_i \leq n$), representing the nodes the i -th edge connects.

Note that the graph can have multiple edges connecting the same two nodes and self-loops. It is guaranteed that the graph is connected.

Output

Output a line containing the lexicographically smallest sequence $a_1, a_2, \dots, a_n$ Bob can record.

Examples

input	output
3 2 1 2 1 3	1 2 3
input	output
5 5 1 4 3 4 5 4 3 2 1 5	1 4 3 2 5

input	output
10 10 1 4 6 8 2 5 3 7 9 4 5 6 3 4 8 10 8 9 1 10	1 4 3 7 9 8 6 5 2 10

Note

In the first sample, Bob's optimal wandering path could be $1 \rightarrow 2 \rightarrow 1 \rightarrow 3$. Therefore, Bob will obtain the sequence $\{1, 2, 3\}$, which is the lexicographically smallest one.

In the second sample, Bob's optimal wandering path could be $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 5$. Therefore, Bob will obtain the sequence $\{1, 4, 3, 2, 5\}$, which is the lexicographically smallest one.

C. Division and Union

time limit per test: 2 seconds
memory limit per test: 256 megabytes

There are n segments $[l_i, r_i]$ for $1 \leq i \leq n$. You should divide all segments into two *non-empty* groups in such way that there is no pair of segments from different groups which have at least one common point, or say that it's impossible to do it. Each segment should belong to exactly one group.

To optimize testing process you will be given multitest.

Input

The first line contains one integer T ($1 \leq T \leq 50000$) — the number of queries. Each query contains description of the set of segments. Queries are independent.

First line of each query contains single integer n ($2 \leq n \leq 10^5$) — number of segments. It is guaranteed that $\sum n$ over all queries does not exceed 10^5 .

The next n lines contains two integers l_i, r_i per line ($1 \leq l_i \leq r_i \leq 2 \cdot 10^5$) — the i -th segment.

Output

For each query print n integers $t_1, t_2, \dots, t_n$ ($t_i \in \{1, 2\}$) — for each segment (in the same order as in the input) t_i equals 1 if the i -th segment will belongs to the first group and 2 otherwise.

If there are multiple answers, you can print any of them. If there is no answer, print -1 .

Example

input	output
3	2 1
2	-1
5 5	1 1 2
2 3	
3	
3 5	
2 3	
2 3	
3	
3 3	
4 4	
5 5	

Note

In the first query the first and the second segments should be in different groups, but exact numbers don't matter.

In the second query the third segment intersects with the first and the second segments, so they should be in the same group, but then the other group becomes empty, so answer is -1 .

In the third query we can distribute segments in any way that makes groups non-empty, so any answer of 6 possible is correct.

C. The Fair Nut and String

time limit per test: 1 second

memory limit per test: 256 megabytes

The Fair Nut found a string s . The string consists of lowercase Latin letters. The Nut is a curious guy, so he wants to find the number of strictly increasing sequences $p_1, p_2, \dots, p_k$, such that:

1. For each i ($1 \leq i \leq k$), $s_{p_i} = 'a'$.
2. For each i ($1 \leq i < k$), there is such j that $p_i < j < p_{i+1}$ and $s_j = 'b'$.

The Nut is upset because he doesn't know how to find the number. Help him.

This number should be calculated modulo $10^9 + 7$.

Input

The first line contains the string s ($1 \leq |s| \leq 10^5$) consisting of lowercase Latin letters.

Output

In a single line print the answer to the problem — the number of such sequences $p_1, p_2, \dots, p_k$ modulo $10^9 + 7$.

Examples

input	output
abbaa	5
input	output
baaaa	4
input	output
agaa	3

Note

In the first example, there are 5 possible sequences. $[1], [4], [5], [1, 4], [1, 5]$.

In the second example, there are 4 possible sequences. $[2], [3], [4], [5]$.

In the third example, there are 3 possible sequences. $[1], [3], [4]$.

C. Cut 'em all!

time limit per test: 1 second
memory limit per test: 256 megabytes

You're given a tree with n vertices.

Your task is to determine the maximum possible number of edges that can be removed in such a way that all the remaining connected components will have even size.

Input

The first line contains an integer n ($1 \leq n \leq 10^5$) denoting the size of the tree.

The next $n - 1$ lines contain two integers u, v ($1 \leq u, v \leq n$) each, describing the vertices connected by the i -th edge.

It's guaranteed that the given edges form a tree.

Output

Output a single integer k — the maximum number of edges that can be removed to leave all connected components with even size, or -1 if it is impossible to remove edges in order to satisfy this property.

Examples

input 4 2 4 4 1 3 1	output 1
input 3 1 2 1 3	output -1
input 10 7 1 8 4 8 10 4 7 6 5 9 3 3 5 2 10 2 5	output 4
input 2 1 2	output 0

Note

In the first example you can remove the edge between vertices 1 and 4. The graph after that will have two connected components with two vertices in each.

In the second example you can't remove edges in such a way that all components have even number of vertices, so the answer is -1 .

C. Nested Segments

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You are given a sequence $a_1, a_2, \dots, a_n$ of one-dimensional segments numbered 1 through n . Your task is to find two distinct indices i and j such that segment a_i lies within segment a_j .

Segment $[l_1, r_1]$ lies within segment $[l_2, r_2]$ iff $l_1 \geq l_2$ and $r_1 \leq r_2$.

Print indices i and j . If there are multiple answers, print any of them. If no answer exists, print -1 -1.

Input

The first line contains one integer n ($1 \leq n \leq 3 \cdot 10^5$) — the number of segments.

Each of the next n lines contains two integers l_i and r_i ($1 \leq l_i \leq r_i \leq 10^9$) — the i -th segment.

Output

Print two distinct indices i and j such that segment a_i lies within segment a_j . If there are multiple answers, print any of them. If no answer exists, print -1 -1.

Examples

input	output
5 1 10 2 9 3 9 2 3 2 9	2 1
input	output
3 1 5 2 6 6 20	-1 -1

Note

In the first example the following pairs are considered correct:

- (2, 1), (3, 1), (4, 1), (5, 1) — not even touching borders;
- (3, 2), (4, 2), (3, 5), (4, 5) — touch one border;
- (5, 2), (2, 5) — match exactly.

B. Minimize the error

time limit per test: 1 second
memory limit per test: 256 megabytes

You are given two arrays A and B , each of size n . The error, E , between these two arrays is defined $E = \sum_{i=1}^n (a_i - b_i)^2$. You have to perform **exactly** k_1 operations on array A and **exactly** k_2 operations on array B . In one operation, you have to choose one element of the array and increase or decrease it by 1.

Output the minimum possible value of error after k_1 operations on array A and k_2 operations on array B have been performed.

Input

The first line contains three space-separated integers n ($1 \leq n \leq 10^3$), k_1 and k_2 ($0 \leq k_1 + k_2 \leq 10^3$, k_1 and k_2 are non-negative) — size of arrays and number of operations to perform on A and B respectively.

Second line contains n space separated integers $a_1, a_2, \dots, a_n$ ($-10^6 \leq a_i \leq 10^6$) — array A .

Third line contains n space separated integers $b_1, b_2, \dots, b_n$ ($-10^6 \leq b_i \leq 10^6$) — array B .

Output

Output a single integer — the minimum possible value of $\sum_{i=1}^n (a_i - b_i)^2$ after doing exactly k_1 operations on array A and exactly k_2 operations on array B .

Examples

input 2 0 0 1 2 2 3	output 2
input 2 1 0 1 2 2 2	output 0
input 2 5 7 3 4 14 4	output 1

Note

In the first sample case, we cannot perform any operations on A or B . Therefore the minimum possible error $E = (1 - 2)^2 + (2 - 3)^2 = 2$.

In the second sample case, we are required to perform exactly one operation on A . In order to minimize error, we increment the first element of A by 1. Now, $A = [2, 2]$. The error is now

$E = (2 - 2)^2 + (2 - 2)^2 = 0$. This is the minimum possible error obtainable.

In the third sample case, we can increase the first element of A to 8, using the all of the 5 moves available to us. Also, the first element of B can be reduced to 8 using the 6 of the 7 available moves.

Now $A = [8, 4]$ and $B = [8, 4]$. The error is now $E = (8 - 8)^2 + (4 - 4)^2 = 0$, but we are still left with 1 move for array B . Increasing the second element of B to 5 using the left move, we get $B = [8, 5]$ and $E = (8 - 8)^2 + (4 - 5)^2 = 1$.

C. Pride

time limit per test: 2 seconds
memory limit per test: 256 megabytes

You have an array a with length n , you can perform operations. Each operation is like this: choose two **adjacent** elements from a , say x and y , and replace one of them with $\gcd(x, y)$, where $\gcd$ denotes the [greatest common divisor](#).

What is the minimum number of operations you need to make all of the elements equal to 1?

Input

The first line of the input contains one integer n ($1 \leq n \leq 2000$) — the number of elements in the array.

The second line contains n space separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array.

Output

Print -1, if it is impossible to turn all numbers to 1. Otherwise, print the minimum number of operations needed to make all numbers equal to 1.

Examples

input 5 2 2 3 4 6	output 5
input 4 2 4 6 8	output -1
input 3 2 6 9	output 4

Note

In the first sample you can turn all numbers to 1 using the following 5 moves:

- [2, 2, 3, 4, 6].
- [2, 1, 3, 4, 6]
- [2, 1, 3, 1, 6]
- [2, 1, 1, 1, 6]
- [1, 1, 1, 1, 6]
- [1, 1, 1, 1, 1]

We can prove that in this case it is not possible to make all numbers one using less than 5 moves.

C. Two TVs

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Polycarp is a great fan of television.

He wrote down all the TV programs he is interested in for today. His list contains n shows, i -th of them starts at moment l_i and ends at moment r_i .

Polycarp owns two TVs. He can watch two different shows simultaneously with two TVs but he can only watch one show at any given moment on a single TV. If one show ends at the same moment some other show starts then you can't watch them on a single TV.

Polycarp wants to check out all n shows. Are two TVs enough to do so?

Input

The first line contains one integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of shows.

Each of the next n lines contains two integers l_i and r_i ($0 \leq l_i < r_i \leq 10^9$) — starting and ending time of i -th show.

Output

If Polycarp is able to check out all the shows using only two TVs then print "YES" (without quotes). Otherwise, print "NO" (without quotes).

Examples

input	output
3 1 2 2 3 4 5	YES
input	output
4 1 2 2 3 2 3 1 2	NO

C. Little Girl and Maximum Sum

time limit per test: 1 second
memory limit per test: 256 megabytes

The little girl loves the problems on array queries very much.

One day she came across a rather well-known problem: you've got an array of n elements (the elements of the array are indexed starting from 1); also, there are q queries, each one is defined by a pair of integers l_i, r_i ($1 \leq l_i \leq r_i \leq n$). You need to find for each query the sum of elements of the array with indexes from l_i to r_i , inclusive.

The little girl found the problem rather boring. She decided to reorder the array elements before replying to the queries in a way that makes the sum of query replies maximum possible. Your task is to find the value of this maximum sum.

Input

The first line contains two space-separated integers n ($1 \leq n \leq 2 \cdot 10^5$) and q ($1 \leq q \leq 2 \cdot 10^5$) — the number of elements in the array and the number of queries, correspondingly.

The next line contains n space-separated integers a_i ($1 \leq a_i \leq 2 \cdot 10^5$) — the array elements.

Each of the following q lines contains two space-separated integers l_i and r_i ($1 \leq l_i \leq r_i \leq n$) — the i -th query.

Output

In a single line print, a single integer — the maximum sum of query replies after the array elements are reordered.

Please, do not use the `%lld` specifier to read or write 64-bit integers in C++. It is preferred to use the `cin`, `cout` streams or the `%I64d` specifier.

Examples

input	output
3 3 5 3 2 1 2 2 3 1 3	25
input	output
5 3 5 2 4 1 3 1 5 2 3 2 3	33